

AMU | Language, Mind, & Tech
Programming in Python
Ali Asgari



Spider\N



Web Scraping



Concept	Human Interface (The Webpage)	Machine Interface (The Scraper)
Visual Hierarchy	Headers, images, and fonts designed to capture human attention.	A nested tree of tags and attributes designed for data extraction.
Data Format	Unstructured, fluid, and often "noisy" with ads and navigation.	Structured CSV, JSON, or SQL databases ready for statistical analysis.
Speed	One page per minute (limited by human reading speed).	Hundreds of pages per second (limited by server politeness).
End Goal	Consumption and understanding.	Systematic evidence for computational research.

Skeleton of Web

(Intro to HTML)



Component	HTML Syntax	Scraping Significance
The Root	<html>	The ultimate ancestor; every piece of data lives inside this "container."
The Body	<body>	Where the actual visible content (and our research data) resides.
Containers	<div> / 	Logical groupings. Think of these as "neighborhoods" where related data lives.
Headers	<h1> to <h6>	The hierarchy of importance. Usually contains the "Title" or "Main Topic".
Links		The "silk threads" that allow your Spider to move from Page A to Page B.
Lists	/	Often used for navigation menus or collections of short research abstracts.
Classes	class="comment"	A "label" applied to many elements. Use this to grab <i>all</i> comments or <i>all</i> titles.
IDs	id="author-42"	A "unique fingerprint." Use this to grab one specific, high-priority data point.
Attributes	src, href, title	The metadata. This is where the "hidden" data (like image URLs) is stored.

Skeleton of Web

(Intro to HTML)



```
# This is the 'Skeleton' our Spider sees
web_skeleton = """
<html>
  <body>
    <div id="research-header">
      <h1 class="title">Linguistics</h1>
      <p class="author">Anna Nowak</p>
    </div>
    <div class="body">
      <p class="data-point">Subject 01: High divergent thinking score.</p>
      <p class="data-point">Subject 02: Average divergent thinking score.</p>
      <a href="https://amu.edu.pl/research" class="external-link">Full Methodology</a>
    </div>
  </body>
</html>
"""
```

Skeleton of Web

(Intro to HTML)



Side Quest!

Why would a researcher prefer to search for a specific **id** rather than a **class** when looking for the "Abstract" of a paper?

Your reward : 1pt

X-Ray

(Browser Inspect Tool)



Feature	Shortcut	Purpose in Research
Open DevTools	Ctrl+Shift+I	Opens the full suite of developer tools used to "X-Ray" the site.
Select Element	Ctrl+Shift+C	The "Sniper Scope." Hover over a piece of text to see its exact HTML home.
View Page Source	Ctrl+U	Shows the "Original Skeleton" as sent by the server (pre-JavaScript).
Refresh & Clear	Ctrl+R	Restarts the "Digital Handshake" to observe how the page loads from scratch.

Tab Name	Digital Function	Research Application
Elements	The Live DOM	Used to find the Tags , Classes , and IDs you will target with your Spider.
Console	The Sandbox	Used to check for errors or to see if data is being "ghosted" in via JavaScript.
Sources	File Directory	Displays the actual scripts and files that build the page's logic and style.
Network	Traffic Monitor	Shows the actual "Data Packets" moving between you and the server.
Application	Local Memory	Shows "Cookies" and local storage (long-term memory).



Component/Method	Syntax	Technical Description
GET Request	<code>requests.get(url)</code>	The standard method for requesting data from a server. It's the digital "Please may I have this page?".
Status Code	<code>response.status_code</code>	An integer representing the server's response (e.g., 200 for success, 404 for not found).
Response Content	<code>response.text</code>	The raw payload of the page, usually returned as a massive string of HTML code.
Headers	<code>response.headers</code>	A dictionary containing metadata about the response, such as content type and server date.



```
import requests

target_url = "https://www.gutenberg.org/cache/epub/11/pg11.txt"
# Alice in Wonderland

# Sending our messenger to the server
print(f"Agent dispatched to: {target_url}...")
response = requests.get(target_url)
```

```
# Checking the server's 'social cues' (Status Code)
if response.status_code == 200:
    print("Success! The server opened the vault.")
    # Capturing the first 250 characters of the text corpus
    corpus_snippet = response.text[:250]
    print("\n--- Data Sample ---")
    print(corpus_snippet)
```

Status Code



Code	Category	The "Scraper" Translation
200	Success	OK. The "Go" signal. The server has opened the doors to the digital laboratory.
301/302	Redirection	Moved. The data has migrated. Your script needs to follow the new URL "thread."
400	Client Error	Bad Request. You sent a garbled message. Check your code's request logic.
401	Client Error	Unauthorized. This room is locked. You need a digital key (login or API token).
403	Client Error	Forbidden. The "Stop" signal. You've been caught being a bot. The server is slamming the door.
404	Client Error	Not Found. The "Stop" signal. You are looking for a page that doesn't exist.
429	Client Error	Too Many Requests. You are scraping too fast! The server is telling you to take a breath.
500	Server Error	Internal Error. The website's "brain" just crashed. It's not your fault, but the data is gone.
503	Server Error	Service Unavailable. The laboratory is undergoing maintenance. Come back later.

Making the Soup

(Introduction to BeautifulSoup)



Method	Syntax	Technical Description
The Parser	BeautifulSoup(markup, 'html.parser')	The "Digestive System." It converts a raw, messy HTML string into a navigable Python object.
Prettifying	soup.prettify()	Re-indent the raw HTML into a human-readable "Visual Skeleton," making it easier to plan your extraction.
Tag Access	soup.tag_name	A "Shortcut" to reach the very first instance of a tag (e.g., soup.h1 or soup.title).
Text Extraction	soup.get_text() or .text	The "Filter." It strips away all HTML tags and returns only the human-readable strings found inside.
Object Type	type(soup)	Confirms you are no longer working with a "String," but a bs4.BeautifulSoup object that understands hierarchy.



```
from bs4 import BeautifulSoup
import requests

url = "https://example.com/neuro-linguistics-report"
# Note: For this demo, we simulate the text we'd get from a website
raw_html = "<html><body><div><h1 id='title'>Neural Correlates </h1><p class='content'>EEG data ...</p></div></body></html>"

# Step 2: Create the Soup Object (The 'Digestive' process) its standard 'html.parser'
soup = BeautifulSoup(raw_html, "html.parser")

# Step 3: Inspect the organized structure
# .prettify() makes the code look 'human' again
print(soup.prettify())

# Step 4: Direct Access
# Grabbing the title directly via the tag name
captured_title = soup.h1.text
print(f"\nExtracted Title: {captured_title}")

# Step 5: Verify we have a smart object, not just a string
print(f"Object Type: {type(soup)}")
```



Side Quest!

What happens if you try to make "Soup" out of a file that isn't HTML (like a raw image or a PDF)?

Your reward : 2pt



Component	Syntax	Technical Description
API Endpoint	<code>https://api.site.com/v1/data</code>	A specific URL designed to serve raw data instead of visual webpages.
JSON	<code>{"key": "value"}</code>	JavaScript Object Notation. A lightweight data-interchange format that Python reads as a Dictionary.
JSON Parser	<code>response.json()</code>	A specialized method that automatically converts a JSON response string into a Python Dictionary/List.
Key-Value Access	<code>data['results'][0]['text']</code>	The method for "drilling down" through nested data structures to find specific linguistic variables.

JSON and APIs



```
import requests

# An API endpoint representing a linguistic database
api_url = "https://api.datamuse.com/words?rel_jja=creative"

# Requesting the raw data from the 'Backdoor'
response = requests.get(api_url)

# If the handshake is successful, we parse the JSON directly
if response.status_code == 200:
    # This turns the response into a Python List of Dictionaries
    word_data = response.json()

    # Accessing the first result in the structured list
    top_result = word_data[0]
    print(f"Top Descriptor: {top_result['word']}")
    print(f"Relevance Score: {top_result['score']}")
else:
    print("API connection failed.")
```

Sorting and Structuring

(Loops & Dictionaries)

AMU | Language, Mind, & Tech | Programming in Python

Ali Asgari



```
from bs4 import BeautifulSoup

html_corpus = """
<div class="paper">
  <h2 class="title">Bilingualism </h2>
  <span class="author">Bartosz</span>
</div>
<div class="paper">
  <h2 class="title">EEG Processing</h2>
  <span class="author">Agnieszka</span>
</div>
"""

soup = BeautifulSoup(html_corpus, 'html.parser')
all_papers = soup.find_all('div', class_='paper')

# This is our empty digital vault
research_database = []

# The Loop: Visiting each 'bone' to build a record
for paper in all_papers:
    # Creating a Dictionary for this specific paper
    entry = {
        "paper_title": paper.find('h2', class_='title').text,
        "lead_researcher": paper.find('span', class_='author').text
    }

    # Adding the jar to our vault
    research_database.append(entry)

# Viewing our structured results
for record in research_database:
    print(f"Topic: {record['paper_title']} | Lead: {record['lead_researcher']}")
```


Cleaning the Noise

(Data Refinement)



Method	Syntax	Technical Description
Whitespace Removal	<code>.strip()</code>	Trims invisible "noise" like spaces, tabs, and newlines from the start and end of a string.
Sub-string Swap	<code>.replace(,)</code>	Targeted surgery. Swaps specific characters or words (e.g., removing "Score: " to leave just the number).
Case Uniformity	<code>.lower()</code> <code>.upper()</code>	Essential for linguistic analysis. Ensures "Cognition" and "cognition" are treated as the same token.
Prefix/Suffix Removal	<code>.removeprefix()</code> <code>.removesuffix()</code>	A clean way to strip away recurring URL parts or specific research labels without affecting the core text.
Type Casting	<code>int()</code> / <code>float()</code>	Converts "scraped strings" into actual numbers so you can perform statistical operations.

```
# The Sanitization Protocol: Cleaning messy linguistic data
raw_scraped_data = " \nRESEARCH_SUBJECT_01: Divergent Thinking "
```

1. Stripping the 'Invisible Mud' (Whitespace and Newlines)

```
clean_step_1 = raw_scraped_data.strip()
```

2. Targeted Surgery: Removing the label

We only want the result, not the 'RESEARCH_SUBJECT_01:' part

```
clean_step_2 = clean_step_1.replace("RESEARCH_SUBJECT_01: ", "")
```

3. Standardization: Lowercasing for linguistic matching

```
final_clean_data = clean_step_2.lower()
```

```
print(f"Original: '{raw_scraped_data}'")
print(f"Cleaned: '{final_clean_data}'")
```

Example of Type Casting for numerical analysis

```
raw_score = "Score: 95.5"
numeric_score = float(raw_score.replace("Score: ", ""))
print(f"Numeric Value for SPSS/R: {numeric_score}")
```

Exporting the Results

(CSV and DataFrames)



Method	Syntax	Technical Description
Pandas DataFrame	<code>pd.DataFrame(list_of_dicts)</code>	It converts your list of dictionaries into a structured, tabular grid.
CSV Export	<code>df.to_csv('name.csv', index=False)</code>	The "Universal Key." Saves your data in a format readable by Excel, SPSS, or R.
Data Inspection	<code>df.head()</code>	Displays the first rows of your dataset to verify the structure before final saving.

```
# The Final Hand-off: From Python memory to a CSV file
import pandas as pd

# Our cleaned research database (from previous steps)
research_data = [
    {"subject_id": 1, "condition": "bilingual", "score": 92.5},
    {"subject_id": 2, "condition": "monolingual", "score": 88.1},
    {"subject_id": 3, "condition": "bilingual", "score": 95.0}
]

# 1. Transform the list into a structured DataFrame
df = pd.DataFrame(research_data)

# 2. Inspect the first few rows to ensure alignment
print("Verifying data structure:")
print(df.head())

# 3. Export to a universal CSV file
# 'index=False' prevents from adding extra column of numbers
df.to_csv('linguistic_results.csv', index=False)
print("\nSuccess! 'linguistic_results.csv' is ready for analysis.")
```

Rules of the Road

(Ethics & Politeness)



Rule	Mechanism	Research Significance
robots.txt	<code>/robots.txt</code>	The digital "No Trespassing" sign. It tells your Spider which directories are off-limits for researchers.
Rate Limiting	<code>time.sleep(seconds)</code>	The "Breath" between requests. Prevents your script from accidentally DDOSing a server and getting your university IP banned.
User-Agent	<code>headers = {'User-Agent': '...'}</code>	The "Digital ID Card." Identifies who you are so webmasters can contact you if your Spider is acting up.
Public vs. Private	PII (Personal Identifiable Info)	The ethical line. Scraping a public news site is research; scraping private DM logs is a legal and ethical disaster.

Rules of the Road

(Ethics & Politeness)



```
import requests
import time
```

1. Identify yourself: Don't be an anonymous 'Python-requests' bot

```
idcard = {
    'User-Agent': 'AMU-Research-Bot/1.0 (researcher@amu.edu.pl)'
}
```

```
target_urls = [
    "https://example.com/page1",
    "https://example.com/page2",
    "https://example.com/page3"
]
```

```
for url in target_urls:
```

```
    print(f"Requesting data from: {url}...")
```

2. Send the request with our ID card

```
    response = requests.get(url, headers=idcard)
```

```
    if response.status_code == 200:
```

```
        print("Data captured successfully.")
```

3. Rate Limiting: we don't overwhelm the server

We wait 2 seconds before the next loop iteration

```
    print("Waiting 2 seconds to be polite...")
```

```
    time.sleep(2)
```

```
print("Research harvest complete.")
```



Side Quest!

Why do some researchers schedule their Spiders to run at 3:00 AM in the server's local time zone? What if a platform has global nature?

Your reward : 2pt